Bikkad IT
Dream Big! Start Small!!! Act Now!!!

Bikkad IT Institute

Seaborn Notes

Part 2 -- Categorical & Multiplot Grids

Compiled by Bikkad IT Institute | Teaching Notes

OUR LOCATIONS

+91 77220 82225	+91 77220 82226	+91 77220 82227	+91 77220 82224
Pimpri-Chinchwad	Chh. Sambhajinagar	Nashik	Hadapsar, Pune

Seaborn Notes -- Part 2

Part 2 of 2: D) Categorical Plots | E) Multiplot Grids

Axes-level vs Figure-level functions, step by step | Prepared for Bikkad IT Institute Students | Level: Beginner | Jupyter Notebook / Google Colab

Topic Index

0. Setup & Meet the Datasets
1. Quick Recap -- Axes-level vs Figure-level

D) Categorical Plots

2. Bar Plot -- barplot() vs catplot(kind='bar')
3. Count Plot -- countplot() vs catplot(kind='count')
4. Box Plot -- boxplot() vs catplot(kind='box')
5. Violin Plot -- violinplot() vs catplot(kind='violin')
6. Swarm Plot -- swarmplot() vs catplot(kind='swarm')

E) Multiplot Grids

7. Joint Plot -- jointplot() (figure-level only)
8. Pair Plot -- pairplot() (figure-level only)
9. Quick Revision Cheat Sheet & Practice Exercises

How to use these notes: every chart is drawn FIRST with its **axes-level** function (one plot on one axes) and THEN with its **figure-level** function (a whole figure that can be split into panels with col=). Each step is a complete, short cell you can copy-paste and run, followed by the real chart it produces. **Lines in bold are new or changed** compared with the previous step. A small comparison table at the end of every chart sums up the difference.

0. Setup & Meet the Datasets

This part uses 2 small, everyday-life datasets (CSV files, also in one Excel workbook). They are deliberately small so students can look at the whole table and connect every dot or bar to a real row. Import the libraries and read the datasets **once** at the top of the notebook -- every example below reuses these DataFrames. sns.set_theme() gives every chart a clean grid style.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style='whitegrid')

food = pd.read_csv('food_orders.csv')
gym = pd.read_csv('gym_members.csv')
```

Dataset 1: food_orders.csv (variable: food)

70 food-app orders: day of the week, Weekday/Weekend, cuisine, Veg/Non-Veg, bill amount, delivery time and the customer's rating. Which cuisine costs the most? Are weekend deliveries slower?

Shape: 70 rows x 8 columns | **Used in:** Bar, Count, Box, Violin, Swarm Plot | **First 5 rows:**

order_id	day	day_type	cuisine	food_type	bill_rs	delivery_min	rating
OD101	Mon	Weekday	Chinese	Veg	400	27	3.7
OD102	Mon	Weekday	Pizza	Non-Veg	520	19	4.1
OD103	Mon	Weekday	Pizza	Non-Veg	510	18	4.6
OD104	Mon	Weekday	Biryani	Non-Veg	390	26	4.0
OD105	Mon	Weekday	Burger	Non-Veg	310	25	4.4

Dataset 2: gym_members.csv (variable: gym)

40 gym members: gender, age, height, weight, daily workout minutes, calories burned and membership type. Taller people weigh more; longer workouts burn more calories.

Shape: 40 rows x 8 columns | **Used in:** Joint Plot, Pair Plot | **First 5 rows:**

member_id	gender	age	height_cm	weight_kg	workout_min	calories_burned	membership
GM201	Male	24	183	77	40	484	Basic
GM202	Male	45	180	81	50	495	Basic
GM203	Male	41	175	68	45	432	Premium
GM204	Male	28	175	69	50	494	Basic
GM205	Male	31	178	81	20	293	Basic

Teaching Note

Before drawing anything, let students run .head(), .shape and .describe() on each dataset and say in their own words what one row means. Seaborn always works the same way: data= the DataFrame, and x= / y= / hue= the column NAMES.

1. Quick Recap -- Axes-level vs Figure-level

A quick reminder before the categorical charts. **Axes-level** functions (barplot, countplot, boxplot, violinplot, swarmplot) draw one plot on one axes. Their figure-level partner is **sns.catplot()** -- choose the chart with kind= and split it into panels with col= or row=. Joint Plot and Pair Plot are figure-level only.

Dataset: food_orders.csv

Syntax

```
sns.boxplot(data=df, x='cat', y='num')                # axes-level
sns.catplot(data=df, x='cat', y='num', kind='box')  # figure-level
kind = 'bar' | 'count' | 'box' | 'violin' | 'swarm' | 'strip'
```

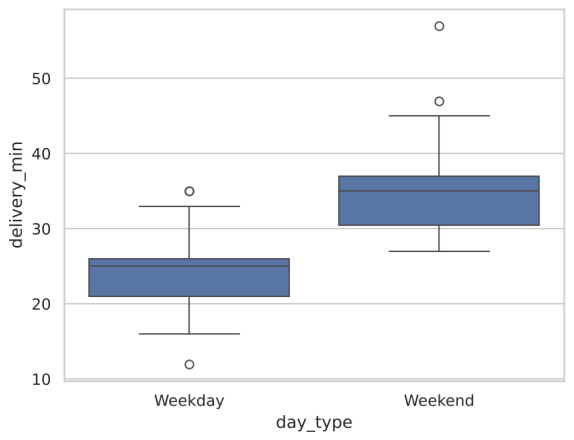
1.1 Axes-level function returns an Axes

```
ax = sns.boxplot(data=food, x='day_type', y='delivery_min')
print(type(ax))
```

Output / Result

```
<class 'matplotlib.axes._axes.Axes'>
```

Output of step 1.1:



Explanation

sns.boxplot() draws on the current axes and returns it -- control the figure with plt.figure(figsize=), plt.title() and so on.

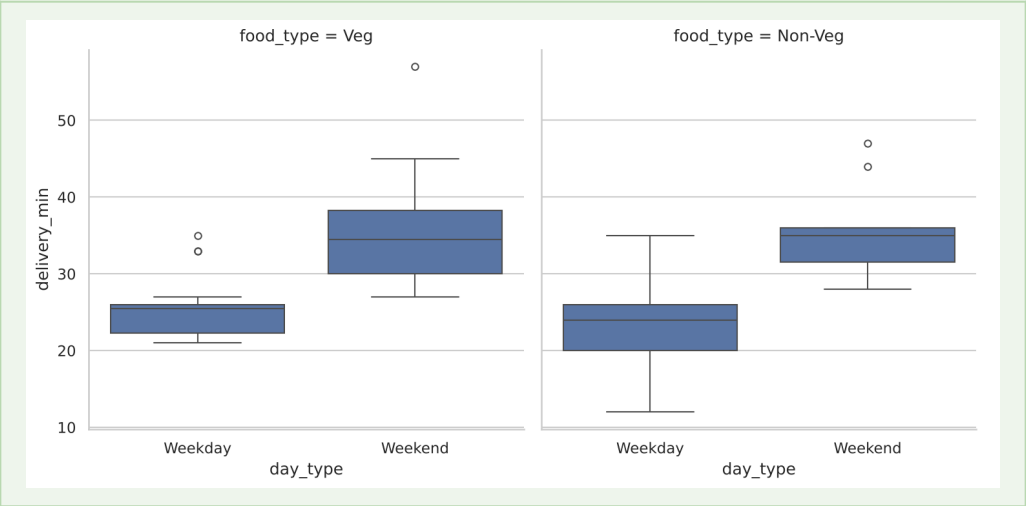
1.2 Figure-level catplot() returns a FacetGrid

```
g = sns.catplot(data=food, x='day_type', y='delivery_min',
                kind='box', col='food_type')
print(type(g))
```

Output / Result

```
<class 'seaborn.axisgrid.FacetGrid'>
```

Output of step 1.2:



Explanation

The same box plot drawn with catplot(kind='box'). It builds its own figure, and col='food_type' splits it into a Veg panel and a Non-Veg panel automatically.

Teaching Note

One figure-level function, sns.catplot(), covers ALL the categorical charts in this part -- just change kind=. Learn the axes-level chart first, then the catplot version is a one-word change.

D) Categorical Plots

2. Bar Plot -- barplot() vs catplot(kind='bar')

Seaborn's bar plot shows the **average of a numeric column for each category** (not the total!) with a small black error bar. Use case: comparing averages -- average bill, average marks, average salary. Our question: which cuisine has the highest average bill?

Dataset: food_orders.csv

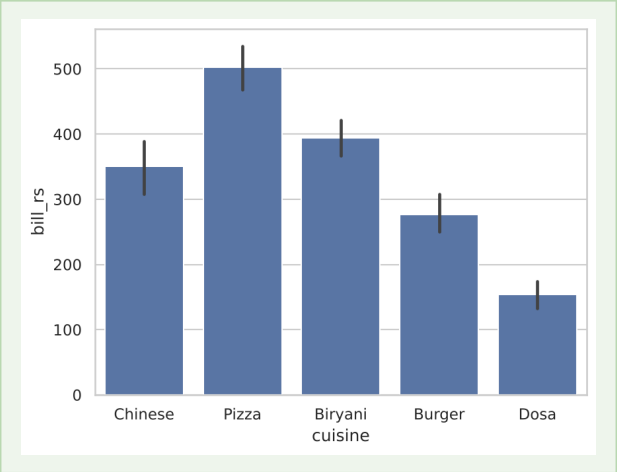
Syntax

```
sns.barplot(data=df, x='cat', y='num', hue=, order=,
            errorbar=, palette=, estimator=)
sns.catplot(data=df, x='cat', y='num', kind='bar', col=)
```

2.1 Axes-level: average bill per cuisine

```
sns.barplot(data=food, x='cuisine', y='bill_rs')
```

Output of step 2.1:



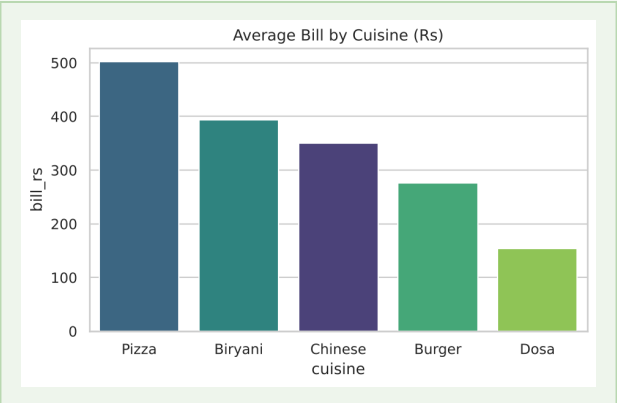
Explanation

One bar per cuisine; the bar height is the AVERAGE bill of that cuisine's orders. The thin black line is the error bar -- how uncertain that average is. Pizza is the most expensive, Dosa the cheapest.

2.2 order=, palette= and errorbar=None

```
plt.figure(figsize=(7, 4))
sns.barplot(data=food, x='cuisine', y='bill_rs',
            order=['Pizza', 'Biryani', 'Chinese', 'Burger', 'Dosa'],
            palette='viridis', hue='cuisine', errorbar=None)
plt.title('Average Bill by Cuisine (Rs)')
```

Output of step 2.2:



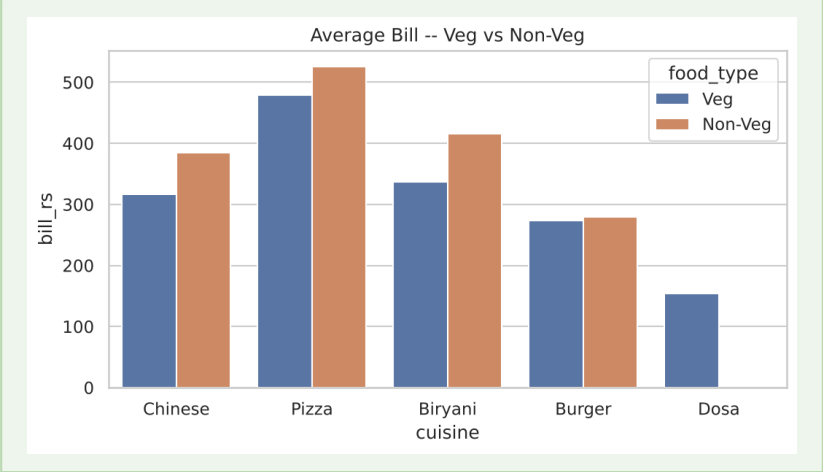
Explanation

order= sets the bar order (highest to lowest here). palette= picks a colour set -- Seaborn now needs hue= set to the same column when you use palette=. errorbar=None hides the error bars.

2.3 hue= -- Veg vs Non-Veg inside each cuisine

```
plt.figure(figsize=(8, 4))
sns.barplot(data=food, x='cuisine', y='bill_rs', hue='food_type',
            errorbar=None)
plt.title('Average Bill -- Veg vs Non-Veg')
```

Output of step 2.3:



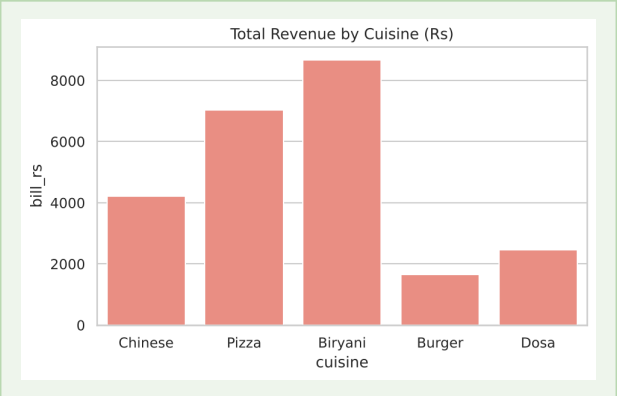
Explanation

hue='food_type' draws two bars side by side for each cuisine (grouped bars) -- one line of code, compared with the np.arange() offsets needed in Matplotlib. Non-Veg orders cost more.

2.4 estimator='sum' -- totals instead of averages

```
plt.figure(figsize=(7, 4))
sns.barplot(data=food, x='cuisine', y='bill_rs', estimator='sum',
            errorbar=None, color='salmon')
plt.title('Total Revenue by Cuisine (Rs)')
```

Output of step 2.4:



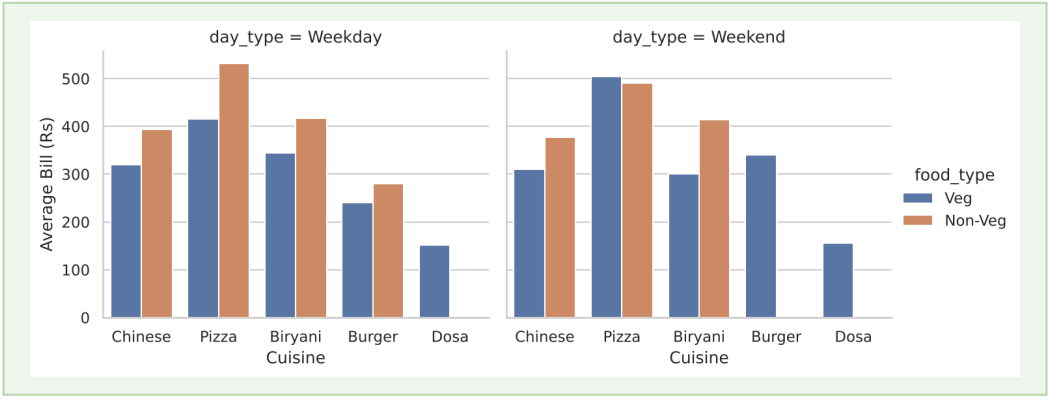
Explanation

estimator='sum' changes what the bar shows: now the TOTAL bill of all orders. Biryani wins on total revenue because it has the most orders, even though Pizza has the higher average.

2.5 Figure-level: catplot(kind='bar') with col=

```
g = sns.catplot(data=food, x='cuisine', y='bill_rs', kind='bar',
                hue='food_type', col='day_type', errorbar=None,
                height=3.8, aspect=1.2)
g.set_axis_labels('Cuisine', 'Average Bill (Rs)')
```

Output of step 2.5:



Explanation

catplot(kind='bar') draws the same grouped bars, and col='day_type' makes one panel for weekdays and one for weekends -- three columns of the data (cuisine, food type, day type) in one figure.

Axes-level vs Figure-level -- for this chart

This chart	Axes-level	Figure-level
Function	`sns.barplot()`	`sns.catplot(kind='bar')`
Bar height	`Average (estimator=)`	`Average (estimator=)`
Size	`plt.figure(figsize=(w, h))`	`height=, aspect=`
Panels	`plt.subplots() + ax=`	`col= / row=`

Teaching Note
The most common beginner mistake: thinking sns.barplot() shows the total. It shows the MEAN by default. Step 4 (estimator='sum') makes the difference unforgettable.

3. Count Plot -- countplot() vs catplot(kind='count')

A count plot is a bar chart of **how many rows** fall in each category -- it counts for you, so there is NO y= column. Use case: number of orders per cuisine, number of students per class, number of customers per city.

Dataset: food_orders.csv

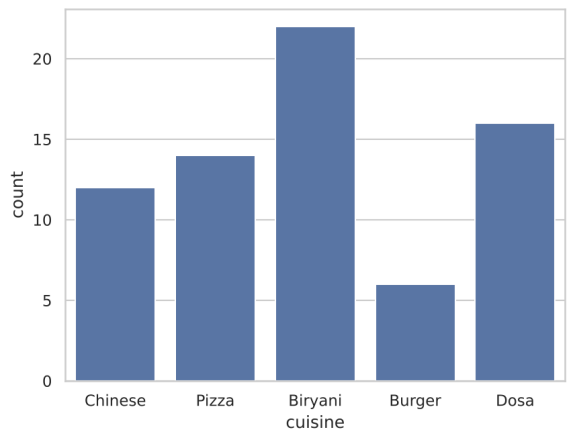
Syntax

```
sns.countplot(data=df, x='cat', hue=, order=, palette=)
sns.countplot(data=df, y='cat')        # horizontal bars
sns.catplot(data=df, x='cat', kind='count', hue=, col=)
```

3.1 Axes-level: orders per cuisine

```
sns.countplot(data=food, x='cuisine')
```

Output of step 3.1:



Explanation

Only x= is given -- Seaborn counts the orders of each cuisine. Biryani is ordered most often.

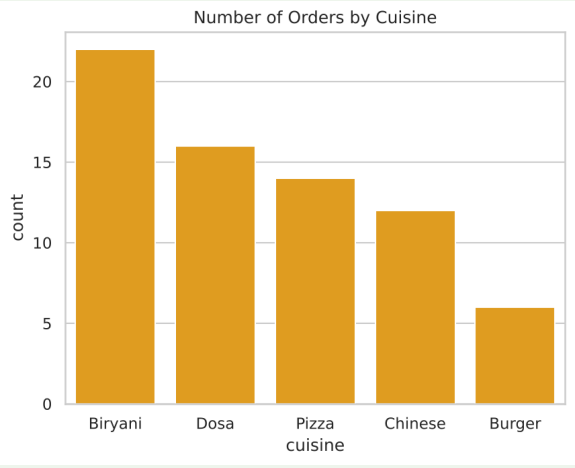
3.2 order= from value_counts() -- most popular first

```
top = food['cuisine'].value_counts().index
print(top)
sns.countplot(data=food, x='cuisine', order=top, color='orange')
plt.title('Number of Orders by Cuisine')
```

Output / Result

```
Index(['Biryani', 'Dosa', 'Pizza', 'Chinese', 'Burger'],
dtype='str', name='cuisine')
```

Output of step 3.2:



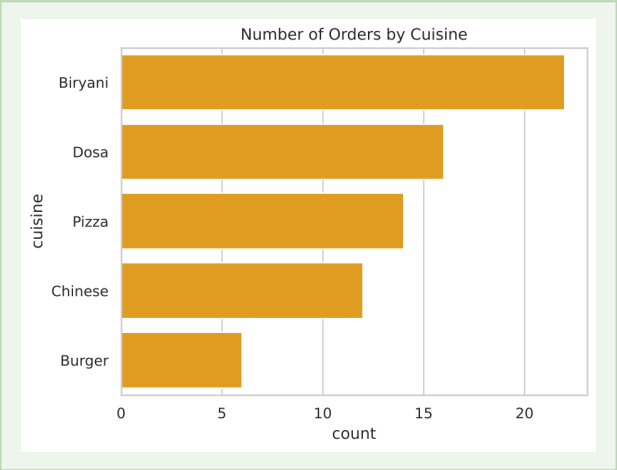
Explanation

value_counts().index gives the cuisines sorted from most to least orders; passing it to order= turns the chart into a ranking.

3.3 y= -- horizontal count plot

```
top = food['cuisine'].value_counts().index
sns.countplot(data=food, y='cuisine', order=top, color='orange')
plt.title('Number of Orders by Cuisine')
```

Output of step 3.3:



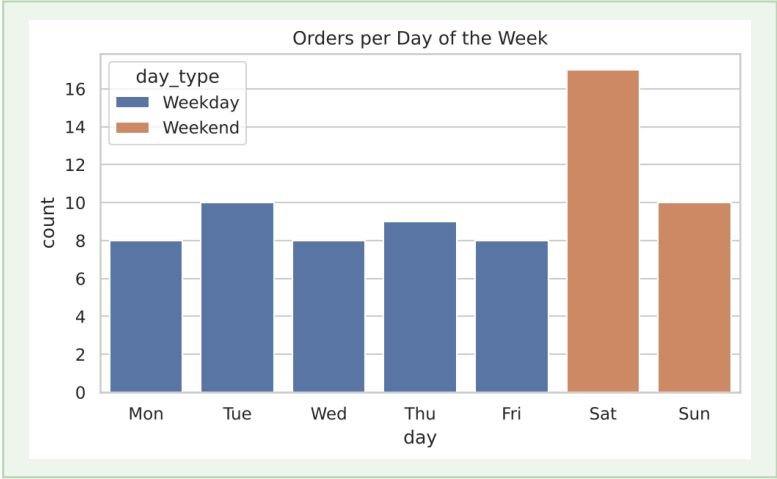
Explanation

Use y= instead of x= and the bars become horizontal -- better for long category names.

3.4 hue= -- weekday vs weekend counts

```
plt.figure(figsize=(7.5, 4))
sns.countplot(data=food, x='day', hue='day_type',
              order=['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun'])
plt.title('Orders per Day of the Week')
```

Output of step 3.4:



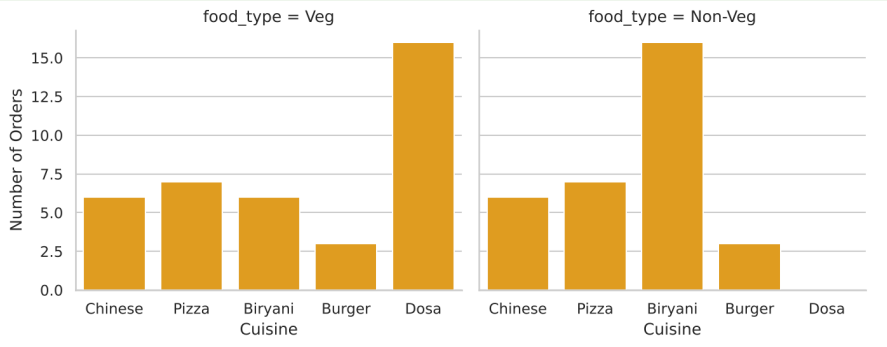
Explanation

order= keeps the days in calendar order and hue='day_type' colours weekend days differently. Saturday is clearly the busiest day of the week.

3.5 Figure-level: catplot(kind='count') with col=

```
g = sns.catplot(data=food, x='cuisine', kind='count',
                col='food_type', color='orange',
                height=3.8, aspect=1.2)
g.set_axis_labels('Cuisine', 'Number of Orders')
```

Output of step 3.5:



Explanation

kind='count' counts rows, just like countplot(). col='food_type' makes a Veg panel and a Non-Veg panel -- Dosa only appears in the Veg panel.

Axes-level vs Figure-level -- for this chart

This chart	Axes-level	Figure-level
Function	`sns.countplot()`	`sns.catplot(kind='count')`
y= needed?	`No -- it counts rows`	`No -- it counts rows`
Size	`plt.figure(figsize=(w, h))`	`height=, aspect=`
Panels	`plt.subplots() + ax=`	`col= / row=`

Teaching Note

Bar plot = average of a number (needs y=). Count plot = how many rows (no y=). Ask students which chart answers 'How many Pizza orders?' and which answers 'How much does a Pizza order cost?'.

4. Box Plot -- boxplot() vs catplot(kind='box')

A box plot summarises a numeric column for each category: the box is the middle 50% of values, the line inside is the **median**, the whiskers show the normal range and separate dots are **outliers**. Use case: comparing the spread of groups and finding unusual values.

Dataset: food_orders.csv

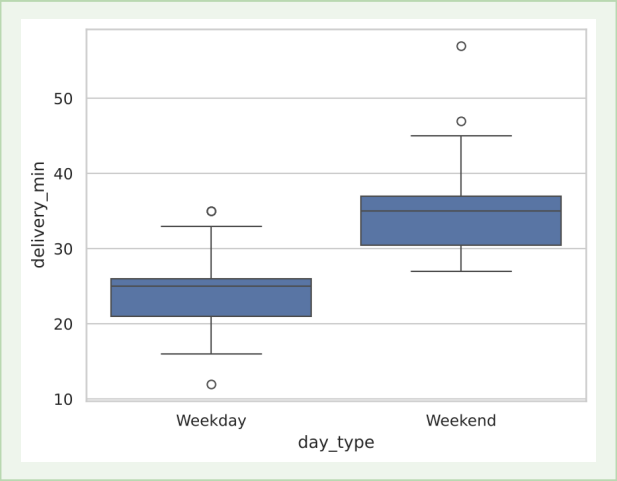
Syntax

```
sns.boxplot(data=df, x='cat', y='num', hue=, order=,
            palette=, width=)
sns.catplot(data=df, x='cat', y='num', kind='box', col=)
```

4.1 Axes-level: delivery time on weekdays vs weekends

```
sns.boxplot(data=food, x='day_type', y='delivery_min')
```

Output of step 4.1:



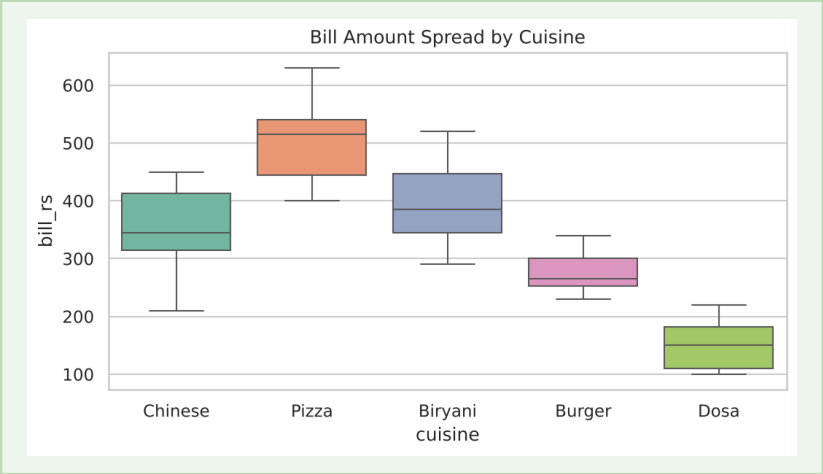
Explanation

Two boxes: the Weekend box sits higher, so weekend deliveries take longer. The dots above the whiskers are a few very late orders (outliers).

4.2 A box for each cuisine + palette=

```
plt.figure(figsize=(8, 4))
sns.boxplot(data=food, x='cuisine', y='bill_rs', hue='cuisine',
            palette='Set2')
plt.title('Bill Amount Spread by Cuisine')
```

Output of step 4.2:



Explanation

Now one box per cuisine. A tall box means the bills of that cuisine vary a lot; a short box means they are similar. Dosa bills are low and consistent.

4.3 hue= -- Veg vs Non-Veg side by side

```
plt.figure(figsize=(8, 4))
sns.boxplot(data=food, x='day_type', y='delivery_min',
            hue='food_type', width=0.6)
plt.title('Delivery Time -- Weekday vs Weekend')
```

Output of step 4.3:



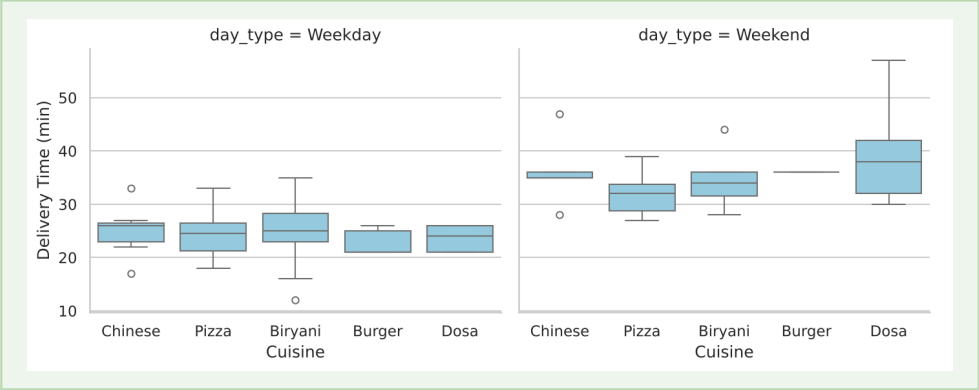
Explanation

hue= splits each box into two coloured boxes. width=0.6 makes the boxes slimmer. The weekend is slower for both Veg and Non-Veg orders.

4.4 Figure-level: catplot(kind='box') with col=

```
g = sns.catplot(data=food, x='cuisine', y='delivery_min',
                kind='box', col='day_type', color='skyblue',
                height=3.8, aspect=1.25)
g.set_axis_labels('Cuisine', 'Delivery Time (min)')
```

Output of step 4.4:



Explanation

The same box plot through catplot(kind='box'). col='day_type' splits weekday and weekend into two panels with the same y-axis scale -- every cuisine is slower at the weekend.

Axes-level vs Figure-level -- for this chart

This chart	Axes-level	Figure-level
Function	`sns.boxplot()`	`sns.catplot(kind='box')`
Size	`plt.figure(figsize=(w, h))`	`height=, aspect=`
Panels	`plt.subplots() + ax=`	`col= / row=`
Labels	`plt.title(), plt.xlabel()`	`g.set_axis_labels()`

Teaching Note

Ask students to find the late orders: food[food['delivery_min'] > 45] -- then point to those exact dots on the box plot.

5. Violin Plot -- violinplot() vs catplot(kind='violin')

A violin plot is a box plot **plus a KDE curve on both sides**. The wider the violin, the more values are there. Use case: seeing the full shape of each group -- not just the median and spread.

Dataset: food_orders.csv

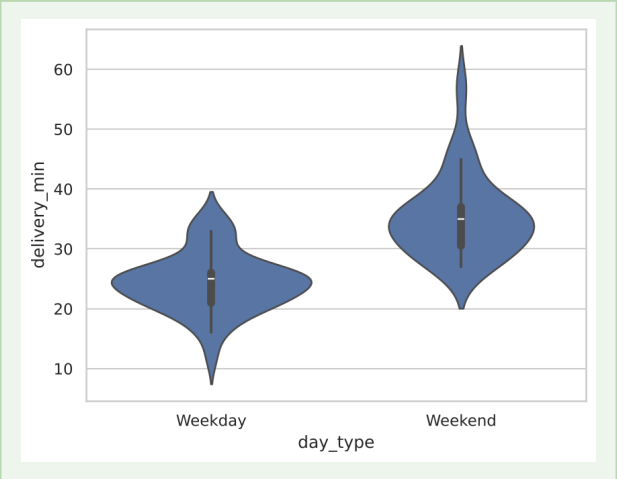
Syntax

```
sns.violinplot(data=df, x='cat', y='num', hue=, split=,
               inner=, palette=)
sns.catplot(data=df, x='cat', y='num', kind='violin', col=)
```

5.1 Axes-level: a plain violin plot

```
sns.violinplot(data=food, x='day_type', y='delivery_min')
```

Output of step 5.1:



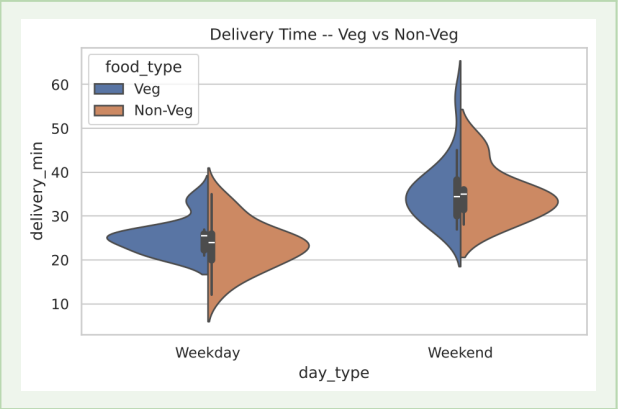
Explanation

Each violin shows where most delivery times lie (the widest part). The tiny box inside is a mini box plot. Weekend violins are higher -- slower deliveries.

5.2 hue= and split=True

```
plt.figure(figsize=(7, 4))
sns.violinplot(data=food, x='day_type', y='delivery_min',
               hue='food_type', split=True)
plt.title('Delivery Time -- Veg vs Non-Veg')
```

Output of step 5.2:



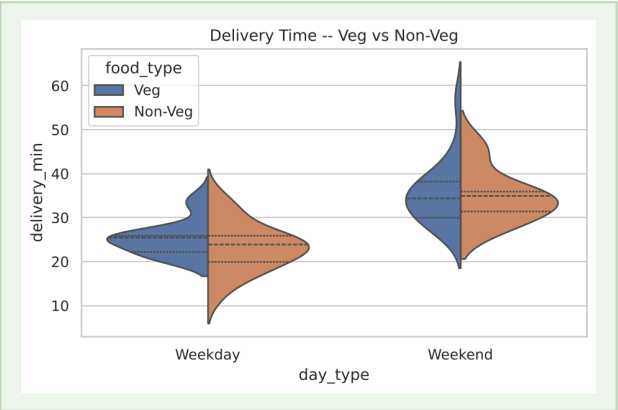
Explanation

With hue= and split=True, each violin is cut in half: left half Veg, right half Non-Veg. Two groups compared in one shape.

5.3 inner='quartile' -- show the quartile lines

```
plt.figure(figsize=(7, 4))
sns.violinplot(data=food, x='day_type', y='delivery_min',
               hue='food_type', split=True, inner='quartile')
plt.title('Delivery Time -- Veg vs Non-Veg')
```

Output of step 5.3:



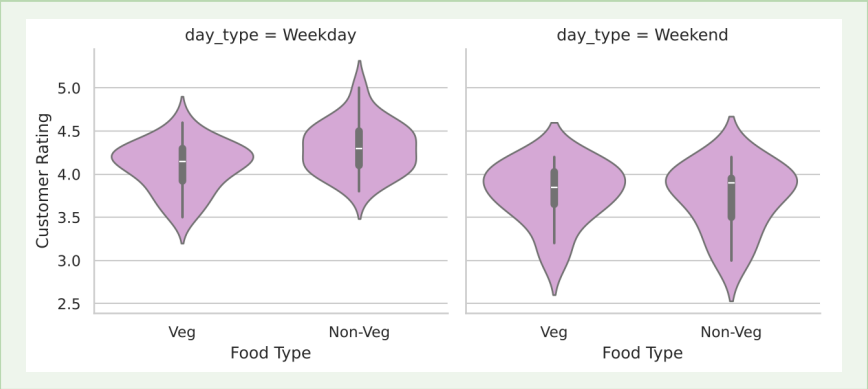
Explanation

inner='quartile' replaces the mini box with three dashed lines: Q1, median and Q3. Try inner='stick' to see every single value.

5.4 Figure-level: catplot(kind='violin') with col=

```
g = sns.catplot(data=food, x='food_type', y='rating',
                kind='violin', col='day_type', color='plum',
                height=3.8, aspect=1.1)
g.set_axis_labels('Food Type', 'Customer Rating')
```

Output of step 5.4:



Explanation

catplot(kind='violin') draws the same chart in a figure-level grid. Ratings are lower at the weekend -- the slow deliveries make customers less happy.

Axes-level vs Figure-level -- for this chart

This chart	Axes-level	Figure-level
Function	`sns.violinplot()`	`sns.catplot(kind='violin')`
Size	`plt.figure(figsize=(w, h))`	`height=, aspect=`
Panels	`plt.subplots() + ax=`	`col= / row=`
split= / inner=	`Yes`	`Yes (same arguments)`

Teaching Note

Put a box plot and a violin plot of the SAME columns next to each other (plt.subplots + ax=). Ask: what can the violin show that the box cannot?

6. Swarm Plot -- swarmplot() vs catplot(kind='swarm')

A swarm plot draws **every single value as a dot** and pushes the dots sideways so that none of them overlap. Use case: small datasets where you want to see each data point -- and where box plots hide how many points there really are.

Dataset: food_orders.csv

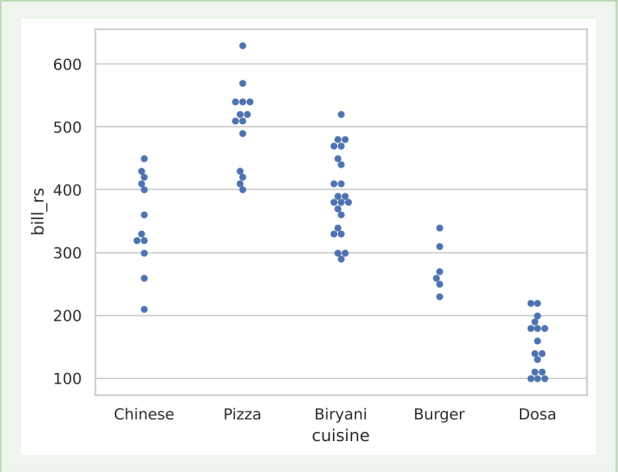
Syntax

```
sns.swarmplot(data=df, x='cat', y='num', hue=, size=,
              palette=)
sns.catplot(data=df, x='cat', y='num', kind='swarm', col=)
```

6.1 Axes-level: every order's bill

```
sns.swarmplot(data=food, x='cuisine', y='bill_rs')
```

Output of step 6.1:



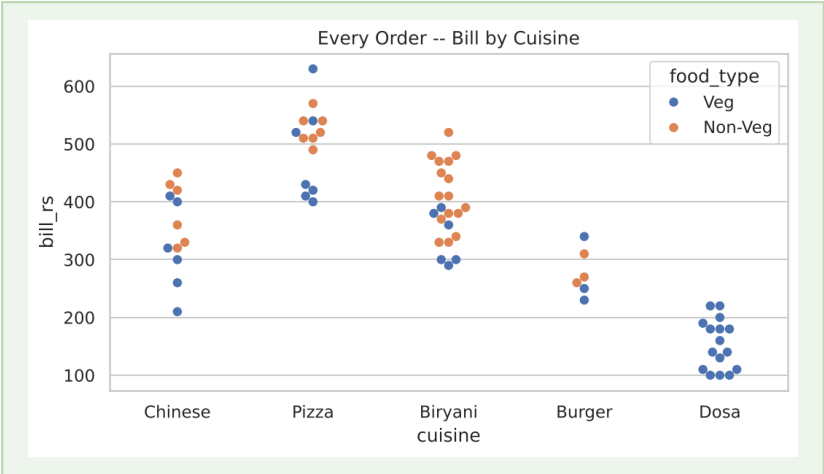
Explanation

70 dots, one per order. The width of each swarm shows how many orders there are: Biryani is wide (many orders), Burger is thin (few orders).

6.2 hue= and size=

```
plt.figure(figsize=(8, 4))
sns.swarmplot(data=food, x='cuisine', y='bill_rs', hue='food_type',
              size=6)
plt.title('Every Order -- Bill by Cuisine')
```

Output of step 6.2:



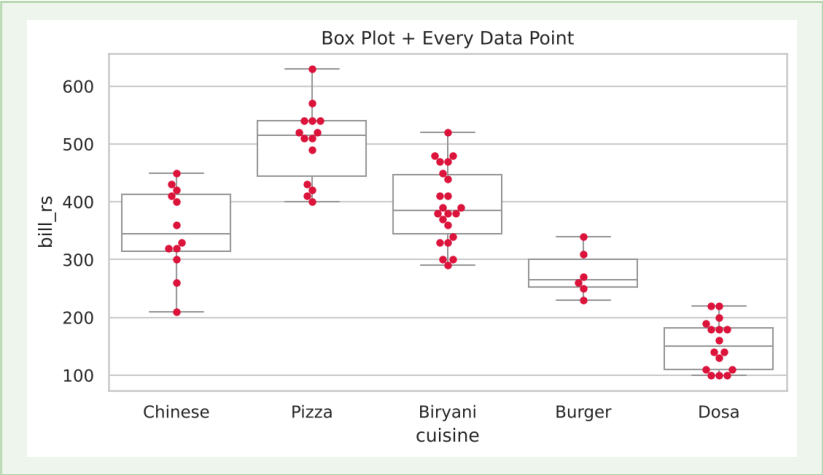
Explanation

hue='food_type' colours each dot Veg or Non-Veg, and size=6 makes the dots bigger. Non-Veg dots sit higher -- they cost more.

6.3 Swarm on top of a box plot

```
plt.figure(figsize=(8, 4))
sns.boxplot(data=food, x='cuisine', y='bill_rs', color='white')
sns.swarmplot(data=food, x='cuisine', y='bill_rs', color='crimson',
              size=5)
plt.title('Box Plot + Every Data Point')
```

Output of step 6.3:



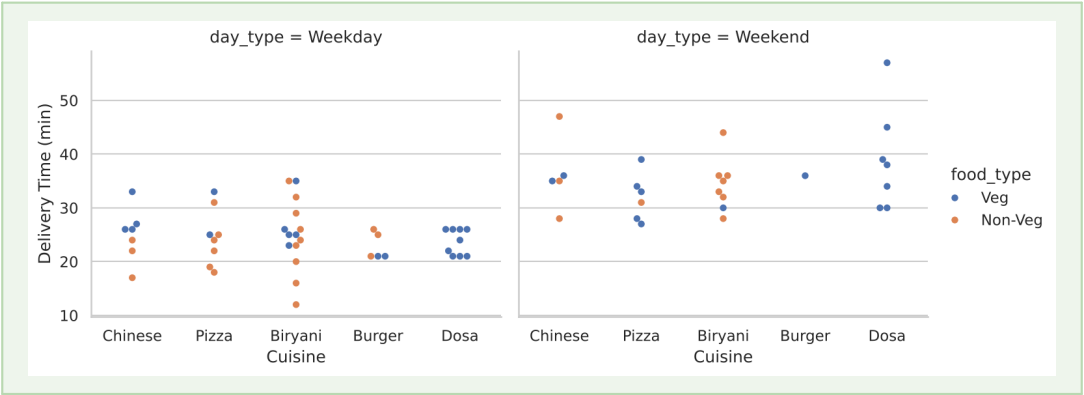
Explanation

Two axes-level functions on the same axes: a white box plot for the summary and red dots for the real values. A favourite chart in reports.

6.4 Figure-level: catplot(kind='swarm') with col=

```
g = sns.catplot(data=food, x='cuisine', y='delivery_min',
                kind='swarm', hue='food_type', col='day_type',
                height=3.8, aspect=1.25)
g.set_axis_labels('Cuisine', 'Delivery Time (min)')
```

Output of step 6.4:



Explanation

catplot(kind='swarm') with col='day_type' -- every order, split by weekday and weekend. Weekend dots are clearly higher.

Axes-level vs Figure-level -- for this chart

This chart	Axes-level	Figure-level
Function	`sns.swarmplot()`	`sns.catplot(kind='swarm')`
Best for	`Small data (up to a few hundred rows)`	`Same, split into panels`
Size	`plt.figure(figsize=(w, h))`	`height=, aspect=`
Panels	`plt.subplots() + ax=`	`col= / row=`

Teaching Note
With thousands of rows a swarm plot becomes too crowded (Seaborn will warn). Then use a box or violin plot, or sns.stripplot() which allows dots to overlap.

E) Multiplot Grids

7. Joint Plot -- jointplot() (figure-level only)

A joint plot shows the **relationship between two numeric columns** in the middle (scatter), and the **distribution of each column** on the top and right edges (histograms). Use case: a full picture of two variables in one figure. jointplot() is **figure-level only** -- it builds a JointGrid, sized with height=.

Dataset: gym_members.csv

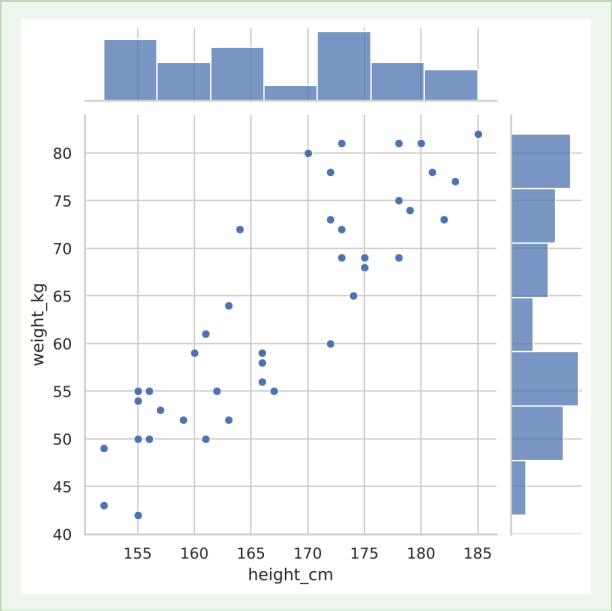
Syntax

```
sns.jointplot(data=df, x='num', y='num', kind=, hue=,
              height=, color=)
kind = 'scatter' | 'reg' | 'hex' | 'kde' | 'hist'
```

7.1 A plain joint plot

```
sns.jointplot(data=gym, x='height_cm', y='weight_kg')
```

Output of step 7.1:



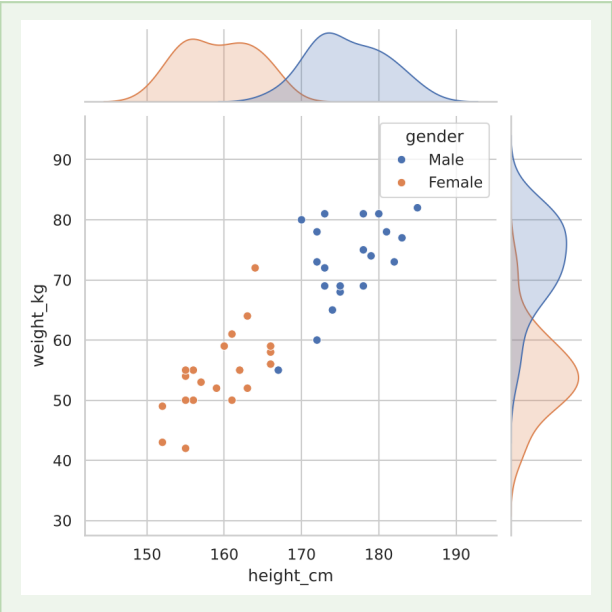
Explanation

Centre: scatter plot of height vs weight -- taller members weigh more. Top: histogram of height. Right: histogram of weight. Three charts from one line.

7.2 hue= -- colour by gender

```
sns.jointplot(data=gym, x='height_cm', y='weight_kg',
              hue='gender', height=6)
```

Output of step 7.2:



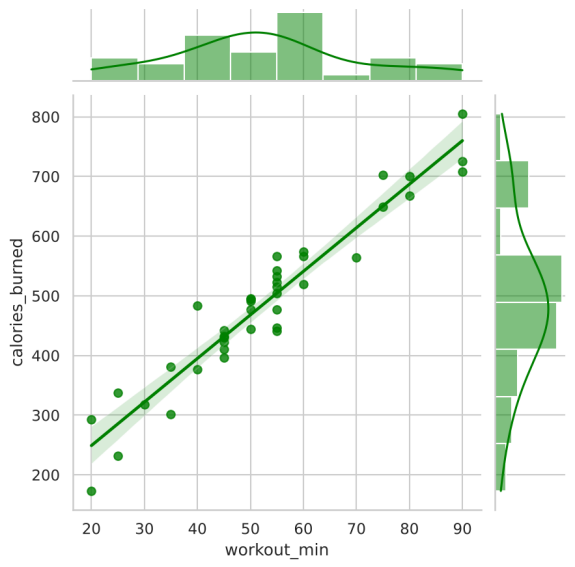
Explanation

hue='gender' colours the dots and turns the edge histograms into KDE curves, one per gender. height=6 sets the figure size (it is always square).

7.3 kind='reg' -- add a trend line

```
sns.jointplot(data=gym, x='workout_min', y='calories_burned',
              kind='reg', color='green', height=6)
```

Output of step 7.3:



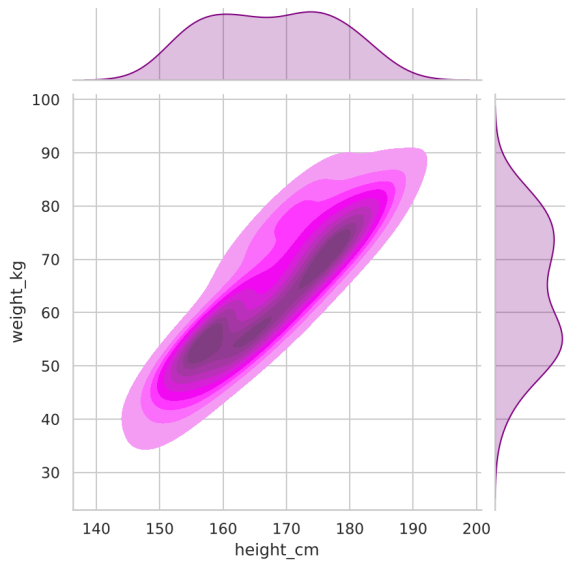
Explanation

kind='reg' adds a best-fit regression line with a shaded band. Longer workouts burn more calories -- a clear upward trend.

7.4 kind='hex' and kind='kde'

```
sns.jointplot(data=gym, x='height_cm', y='weight_kg',
              kind='kde', fill=True, color='purple', height=6)
```

Output of step 7.4:



Explanation

kind='kde' draws smooth density 'contours' instead of dots -- the darkest area is where most members are. Try kind='hex' too: hexagons that are darker where more members fall.

Axes-level vs Figure-level -- for this chart

This chart	Axes-level	Figure-level
Function	`-- (no axes-level version)`	`sns.jointplot()`
Size	`--`	`height= (always square)`
Shows	`--`	`Relationship + both distributions`
ax= allowed	`--`	`No -- always a new figure`

Teaching Note
Ask students: from the joint plot, what is the most common height range? Read it from the top histogram, not the scatter.

8. Pair Plot -- pairplot() (figure-level only)

A pair plot draws a **scatter plot for EVERY pair of numeric columns** in a grid, with each column's distribution on the diagonal. Use case: the first step of EDA -- see all the relationships in a dataset at once. pairplot() is **figure-level only** (a PairGrid).

Dataset: gym_members.csv

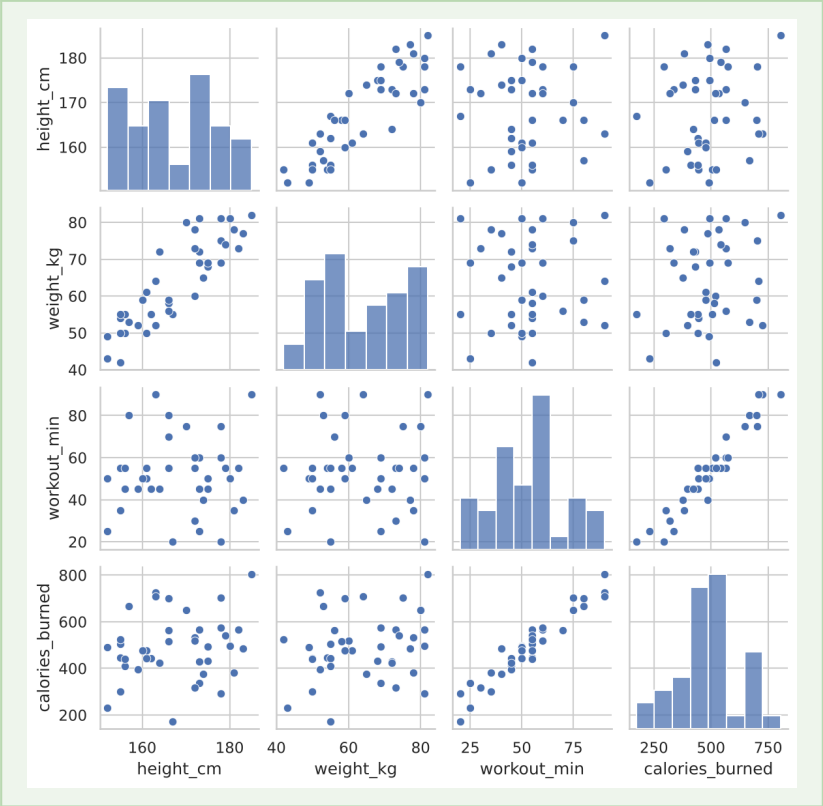
Syntax

```
sns.pairplot(data=df, vars=[...], hue=, kind=, diag_kind=,
             corner=, height=)
```

8.1 A plain pair plot of 4 columns

```
sns.pairplot(data=gym, vars=['height_cm', 'weight_kg',
                             'workout_min', 'calories_burned'],
             height=2)
```

Output of step 8.1:



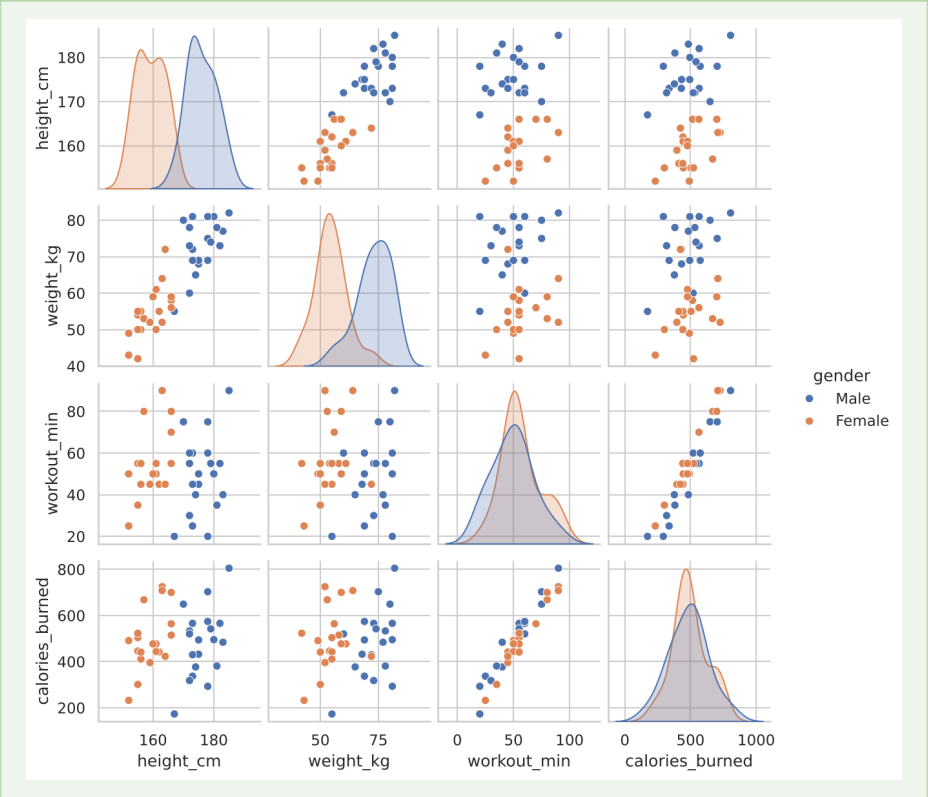
Explanation

vars= picks the columns (without it, ALL numeric columns are used). 4 columns give a 4 x 4 grid: scatter plots off the diagonal and histograms on the diagonal. height=2 is the size of each small panel.

8.2 hue= -- colour by gender

```
sns.pairplot(data=gym, vars=['height_cm', 'weight_kg',
                             'workout_min', 'calories_burned'],
             hue='gender', height=2)
```

Output of step 8.2:



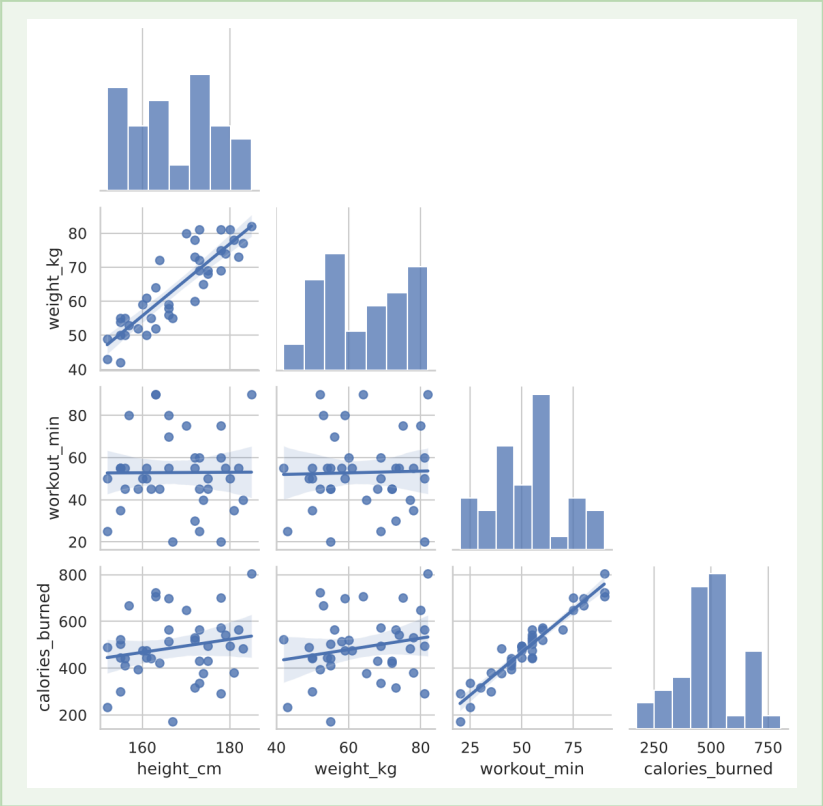
Explanation

hue='gender' colours every scatter plot and changes the diagonal into KDE curves per gender. Height and weight clearly separate male and female members.

8.3 corner=True and kind='reg'

```
sns.pairplot(data=gym, vars=['height_cm', 'weight_kg',
                             'workout_min', 'calories_burned'],
             kind='reg', corner=True, height=2)
```

Output of step 8.3:



Explanation

The top half of a pair plot repeats the bottom half (mirror image). corner=True removes it. kind='reg' adds a trend line to every scatter plot, making strong relationships easy to spot.

Axes-level vs Figure-level -- for this chart

This chart	Axes-level	Figure-level
Function	`-- (no axes-level version)`	`sns.pairplot()`
Size	`--`	`height= per small panel`
Shows	`--`	`All pairs of columns in one grid`
ax= allowed	`--`	`No -- always a new figure`

Teaching Note

Ask students to name the two STRONGEST relationships in the pair plot, then confirm them with a correlation heatmap: sns.heatmap(gym[cols].corr(), annot=True).

9. Quick Revision Cheat Sheet

Category	Chart	Axes-level	Figure-level
D) Categorical	Bar Plot	`sns.barplot()`	`sns.catplot(kind='bar')`
D) Categorical	Count Plot	`sns.countplot()`	`sns.catplot(kind='count')`
D) Categorical	Box Plot	`sns.boxplot()`	`sns.catplot(kind='box')`
D) Categorical	Violin Plot	`sns.violinplot()`	`sns.catplot(kind='violin')`
D) Categorical	Swarm Plot	`sns.swarmplot()`	`sns.catplot(kind='swarm')`
E) Multiplot	Joint Plot	`--`	`sns.jointplot()`
E) Multiplot	Pair Plot	`--`	`sns.pairplot()`

Key Differences

Point	Axes-level	Figure-level
What it draws	One plot on one Matplotlib axes	A whole figure (FacetGrid / JointGrid / PairGrid / ClusterGrid)
Use inside plt.subplots()	Yes, with ax=	No -- always a new figure
Figure size	plt.figure(figsize=(w, h))	height= and aspect=
Split into panels	Not built in	col=, row=, col_wrap=
Legend position	Inside the plot	Outside, on the right
Returns	Matplotlib Axes	Seaborn grid object
Labels	plt.title(), plt.xlabel()	g.set_axis_labels(), g.set_titles()

Practice Exercises (try these on your own)

1. **Bar:** average rating per cuisine with hue='day_type'.
2. **Count:** orders per food_type with catplot(kind='count', col='day_type').
3. **Box:** boxplot of rating by day -- which day has the lowest median rating?
4. **Violin:** violinplot of calories_burned by membership with hue='gender', split=True.
5. **Swarm:** swarmplot of workout_min by gender on top of a box plot.
6. **Joint/Pair:** jointplot of age vs calories_burned (kind='reg'), then a pairplot of the food data with hue='day_type'.